COMPSCI 683: Artificial Intelligence

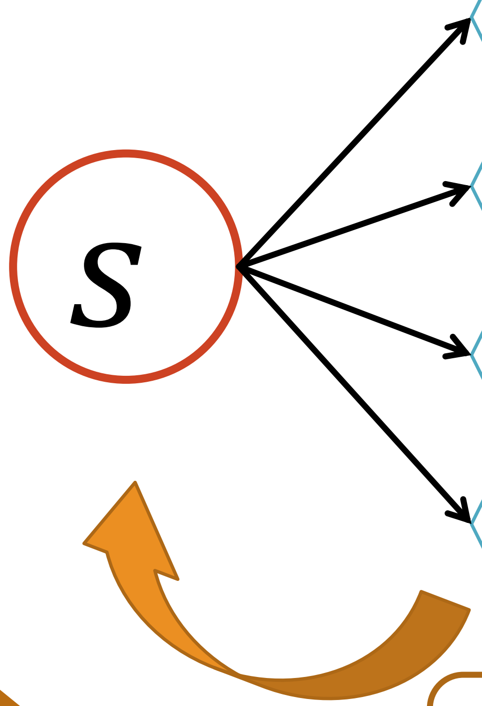
Instructor: Yair Zick

Regret Minimization

"Hoping for the best but expecting the worst"

- Alphaville, *Forever Young*

Choosing the Best Action at a State



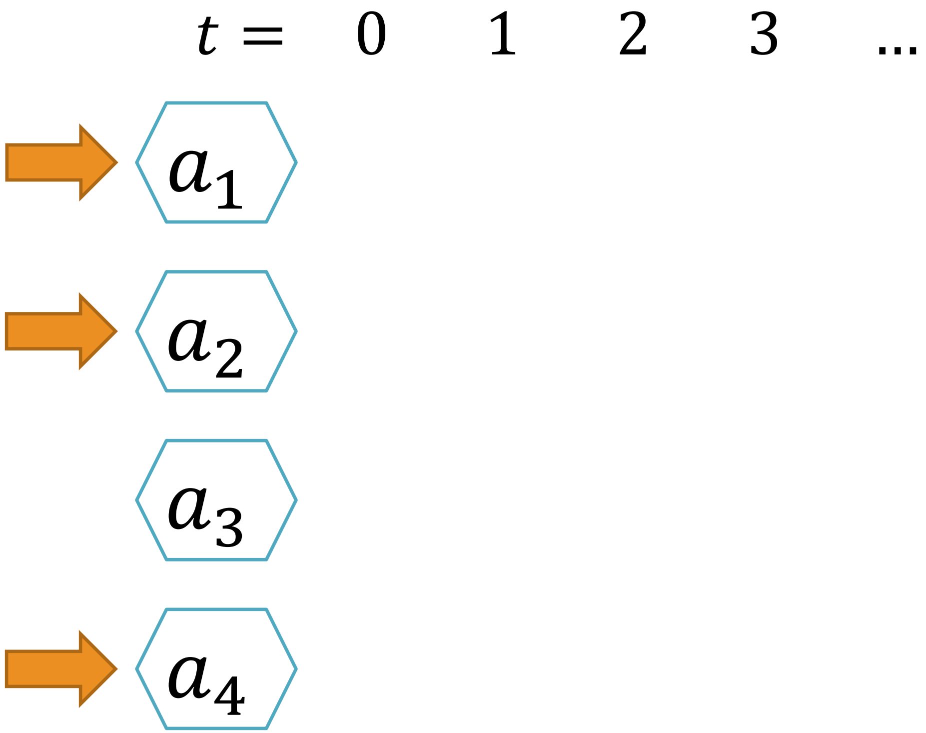
$t =$	0	1	2	3	...
a_1	0	0	1	1	...
a_2	0	1	0	0	...
a_3	0	1	1	1	...
a_4	1	1	0	1	...

We assume 0/1 rewards

Reward depends on:
- Current state s
- Action taken
- The environment
- A terrible person who **wants** us to lose!

For simplicity, single-state domain. All actions return to s.

Multi-Armed Bandit Problem



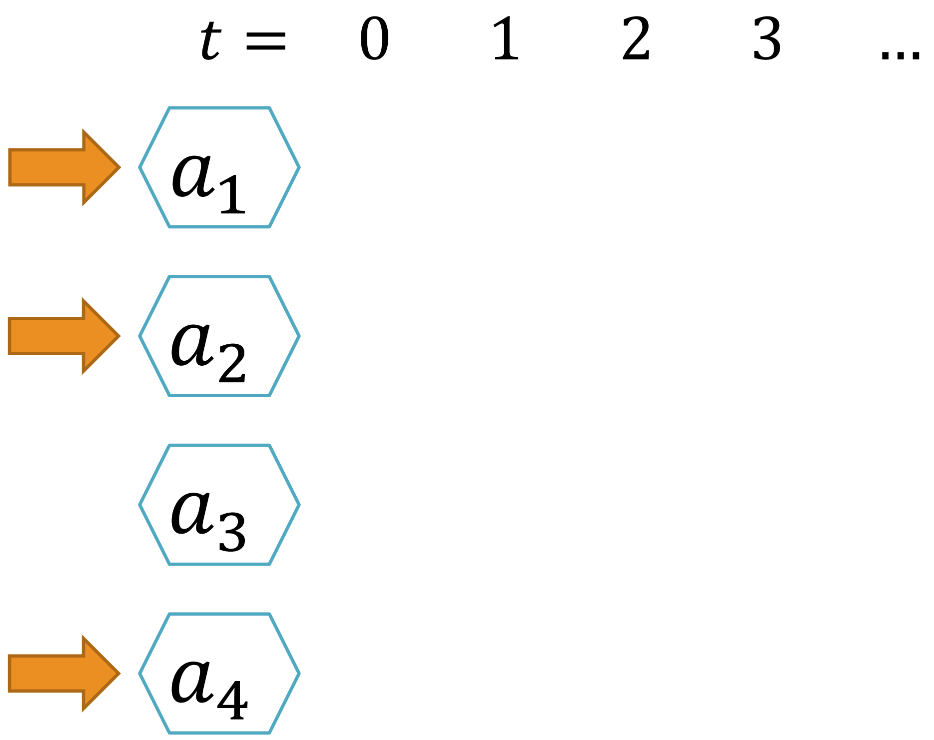
Classic Multi-Armed Bandits (MAB):

- Single state, take actions (that return to the same state)
- Actions offer $0/1$ rewards
- Only observe reward from action chosen, not others.

Goal

Find optimal policy/best action

Expert Systems



Will it rain tomorrow?

How do I divide my money between these four stocks?

Classic Expert Systems:

- Single state, take actions (that return to the same state)
- Actions offer $0/1$ rewards
- Observe rewards from all actions, irrespective of what we picked.

Goal

Find optimal policy

Regret

We want to do well – benchmark against something!

1. Do at least as well as the best algorithm?
2. Do at least as well as the best action?

Definitions

- A policy π assigns a probability p_i^t of playing action i at time t.
- Value of π is $v_\pi^t = \sum_i p_i^t v_i^t$ - the expected reward at time t.
- $V_\pi^t = \sum_{\tau=1}^t v_\pi^\tau$: total reward up to time t.

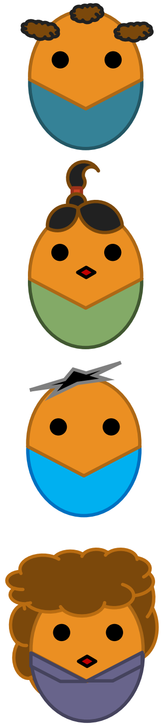
Regret – Compare to Best Action

- Average reward of π vs. average reward of best action: $\frac{1}{\mathrm{T}} \times (V_{best}^t - V_\pi^t)$

Round 1 – Greedy algorithm

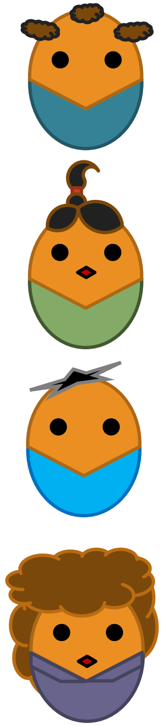
- At round t, let S_t be the set of actions that maximize total reward up to time t.
- Pick one of the best actions (choose the one with the **lowest index** if $|S_t| \geq 2$).

… or any other **deterministic** tie-breaking scheme. Tie breaking scheme can even depend on the history.



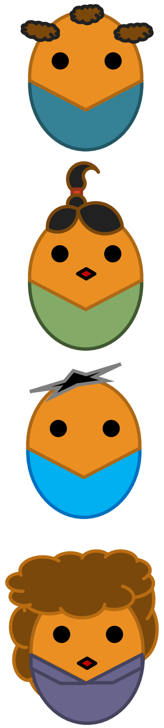
?	?	?	?	$V_1^{t-1} = 0$
?	?	?	?	$V_2^{t-1} = 0$
?	?	?	?	$V_3^{t-1} = 0$
?	?	?	?	$V_4^{t-1} = 0$

$$V_{Greedy}^0 = 0$$
$$V_{best}^0 = 0$$



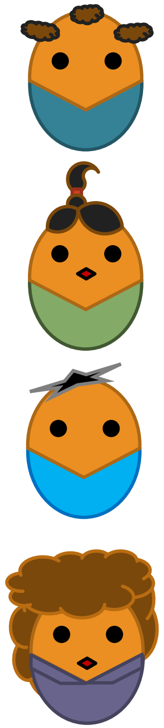
0	?	?	?	$V_1^{t-1} = 0$
1	?	?	?	$V_2^{t-1} = 0$
0	?	?	?	$V_3^{t-1} = 0$
1	?	?	?	$V_4^{t-1} = 0$

$$V_{Greedy}^{1} = 0$$
$$V_{best}^{1} = 1$$



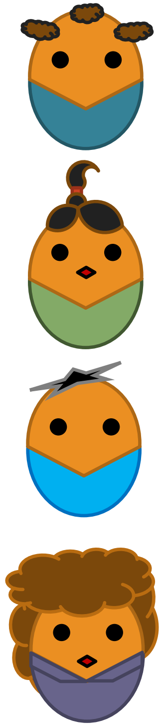
0	?	?	?	$V_1^{t-1} = 0$
1	?	?	?	$V_2^{t-1} = 1$
0	?	?	?	$V_3^{t-1} = 0$
1	?	?	?	$V_4^{t-1} = 1$

$$V_{Greedy}^1 = 0$$
$$V_{best}^1 = 1$$



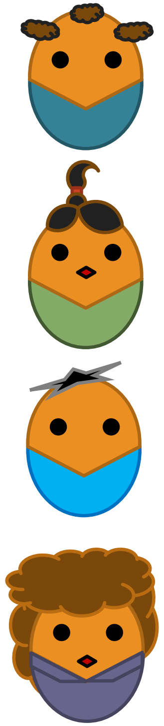
0	1	?	?	$V_1^{t-1} = 0$
1	0	?	?	$V_2^{t-1} = 1$
0	1	?	?	$V_3^{t-1} = 0$
1	1	?	?	$V_4^{t-1} = 1$

$$V_{Greedy}^2 = 0$$
$$V_{best}^2 = 2$$



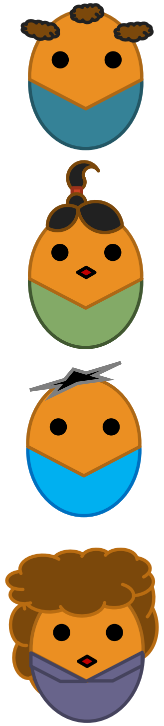
0	1	?	?	$V_1^{t-1} = 1$
1	0	?	?	$V_2^{t-1} = 1$
0	1	?	?	$V_3^{t-1} = 1$
1	1	?	?	$V_4^{t-1} = 2$

$$V_{Greedy}^2 = 0$$
$$V_{best}^2 = 2$$



0	1	1	?	$V_1^{t-1} = 1$
1	0	0	?	$V_2^{t-1} = 1$
0	1	1	?	$V_3^{t-1} = 1$
1	1	0	?	$V_4^{t-1} = 2$

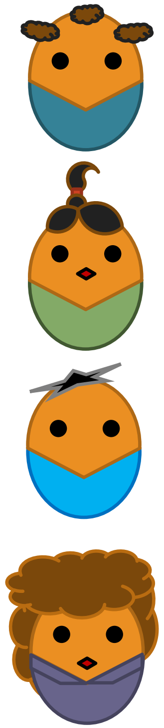
$$V_{Greedy}^{3} = 0$$
$$V_{best}^{3} = 2$$



0	1	1	?	$V_1^{t-1} = 2$
1	0	0	?	$V_2^{t-1} = 1$
0	1	1	?	$V_3^{t-1} = 2$
1	1	0	?	$V_4^{t-1} = 2$

$$V_{Greedy}^{3} = 0$$

$$V_{best}^{3} = 2$$



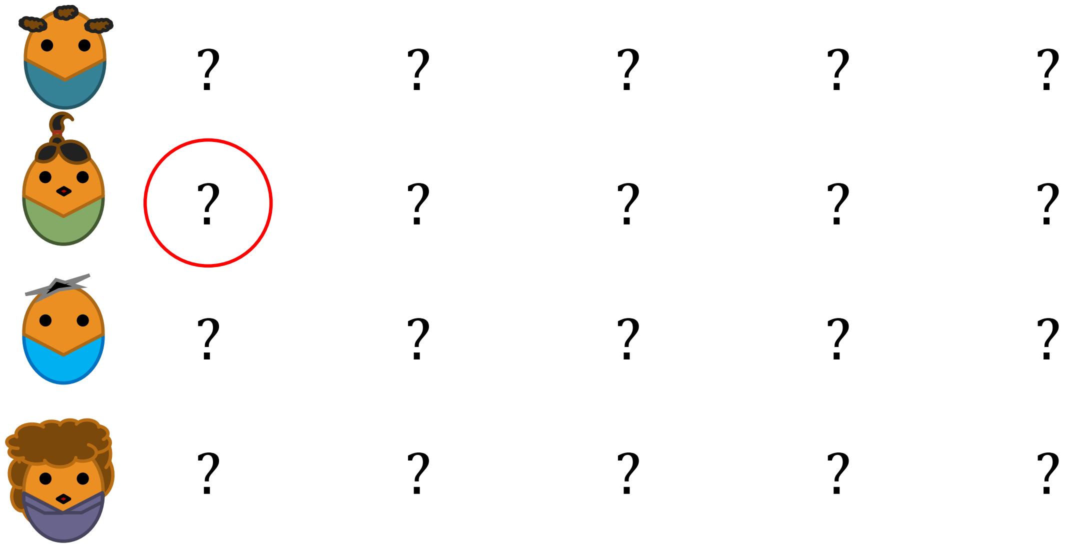
0	1	1	0	$V_1^{t-1} = 2$
1	0	0	1	$V_2^{t-1} = 1$
0	1	1	0	$V_3^{t-1} = 2$
1	1	0	1	$V_4^{t-1} = 3$

$$V_{Greedy}^{4} = 0$$

$$V_{best}^{4} = 3$$

Theorem:
for any **deterministic** algorithm $\mathcal{A}$, there is a sequence of actions for which $V_{best}^{T} \geq \left\lceil \frac{(n-1)\times T}{n} \right\rceil$ but $V_{\mathcal{A}}^{T} = 0$.

Proof:



Theorem:
for any **deterministic** algorithm $\mathcal{A}$, there is a sequence of actions for which $V_{best}^{T} \geq \left\lceil \frac{(n-1)\times T}{n} \right\rceil$ but $V_{\mathcal{A}}^{T} = 0$.

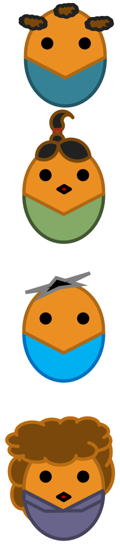
Proof:

1	?	?	?	?
0	?	?	?	?
1	?	?	?	?
1	?	?	?	?

Theorem:

for any **deterministic** algorithm $\mathcal{A}$, there is a sequence of actions for which $V_{best}^{T} \geq \left\lceil \frac{(n-1) \times T}{n} \right\rceil$ but $V_{\mathcal{A}}^{T} = 0$.

Proof:



1	?	?	?	?
0	?	?	?	?
1	?	?	?	?
1	?	?	?	?

Theorem:
for any **deterministic** algorithm $\mathcal{A}$, there is a sequence of actions for which $V_{best}^{T} \geq \left\lceil \frac{(n-1)\times T}{n} \right\rceil$ but $V_{\mathcal{A}}^{T} = 0$.

Proof:

	1	1	?	?	?
	0	1	?	?	?
	1	1	?	?	?
	1	0	?	?	?

Theorem:
for any **deterministic** algorithm $\mathcal{A}$, there is a sequence of actions for which $V_{best}^{T} \geq \left\lceil \frac{(n-1)\times T}{n} \right\rceil$ but $V_{\mathcal{A}}^{T} = 0$.

Proof:

	1	1	0	?	?
	0	1	1	?	?
	1	1	1	?	?
	1	0	1	?	?

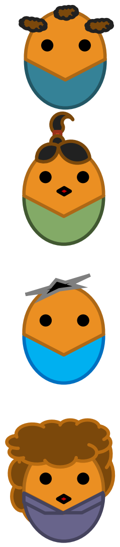
Theorem:
for any **deterministic** algorithm $\mathcal{A}$, there is a sequence of actions for which $V_{best}^{T} \geq \left\lceil \frac{(n-1)\times T}{n} \right\rceil$ but $V_{\mathcal{A}}^{T} = 0$.

Proof:

	1	1	0	?	?
	0	1	1	?	?
	1	1	1	?	?
	1	0	1	?	?

Theorem:
for any **deterministic** algorithm $\mathcal{A}$, there is a sequence of actions for which $V_{best}^{T} \geq \left\lceil \frac{(n-1)\times T}{n} \right\rceil$ but $V_{\mathcal{A}}^{T} = 0$.

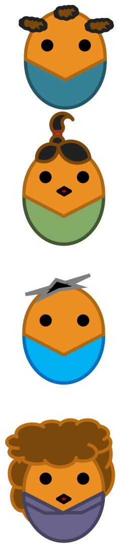
Proof:



1	1	0	1	?
0	1	1	1	?
1	1	1	0	?
1	0	1	1	?

Theorem:
for any **deterministic** algorithm $\mathcal{A}$, there is a sequence of actions for which $V_{best}^{T} \geq \left\lceil \frac{(n-1)\times T}{n} \right\rceil$ but $V_{\mathcal{A}}^{T} = 0$.

Proof:



1	1	0	1	0
0	1	1	1	1
1	1	1	0	1
1	0	1	1	1

Theorem:
for any **deterministic** algorithm $\mathcal{A}$, there is a sequence of actions for which $V_{best}^{T} \geq \left\lceil\frac{(n-1)\times T}{n}\right\rceil$ but $V_{\mathcal{A}}^{T} = 0$.

We constructed an $n \times T$ matrix, with one 0 in each column.

Average number of 1s in each row?

$$n\left\{\overbrace{\begin{matrix} 1 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 & 1 \end{matrix}}^{T}\right.$$

Theorem:
for any **deterministic** algorithm $\mathcal{A}$, there is a sequence of actions for which $V_{best}^{T} \geq \left\lceil \frac{(n-1)\times T}{n} \right\rceil$ but $V_{\mathcal{A}}^{T} = 0$.

We constructed **adversarial input**. Best action is "almost perfect".

Fully predictable algorithms can be exploited by adversaries!

Round 2 – Randomized Greedy

```
Initialize V_i^0 = 0 for all i ∈ N;
At time t = 1,…,∞:
    V_max^t ← max_i V_i^(t−1);
    S_t ← {i ∈ N : V_i^(t−1) = V_max^t};
    For i ∈ N do:
        If i ∈ S_t
            p_i^t ← 1/|S_t|;
        Else
            p_i^t ← 0;
        End If
    End For
```

Initialize $V_i^0 = 0$ for all $i \in N$;

At time $t = 1, \ldots, \infty$:

$V_{\max}^t \leftarrow \max_i V_i^{t-1}$;

$S_t \leftarrow \{i \in N : V_i^{t-1} = V_{\max}^t\}$; — Compute set of best actions S_t

For $i \in N$ **do**:

If $i \in S_t$

$p_i^t \leftarrow \frac{1}{|S_t|}$; — Pick one of the best actions with prob. $\frac{1}{|S_t|}$

Else

$p_i^t \leftarrow 0$; — The rest are never picked

End If

End For

Round 2 – Randomized Greedy

The RG algorithm can lose at most $\log n + 1$ times (in expectation) for every loss of the best action.

Key Insight 1: when some actions from the best actions S_t lose, we remove them from the set of best actions.

Key Insight 2: for any fraction $\frac{a}{b}$ where $a, b \in \mathbb{Z}_+$,

$$\frac{a}{b} \leq \frac{1}{b} + \frac{1}{b-1} + \frac{1}{b-2} + \cdots + \frac{1}{b-a+1}$$

Round 2 – Randomized Greedy

Key Insight 2: for any fraction $\frac{a}{b}$ where $a, b \in \mathbb{Z}_+$,

$$\frac{a}{b} = \underbrace{\frac{1}{b} + \cdots + \frac{1}{b}}_{a \text{ times}} \leq \frac{1}{b} + \frac{1}{b-1} + \frac{1}{b-2} + \cdots + \frac{1}{b-a+1}$$

Round 2 – Randomized Greedy

The RG algorithm can lose at most $\log n + 1$ times (in expectation) for every loss of the best action.

For the next few slides – losses instead of rewards: having a value of 1 is bad!

Round 2 – Randomized Greedy

Suppose that the best action loses at time-steps $t_1, \ldots, t_\ell$ (in other words at time t_j the best action lost j times).

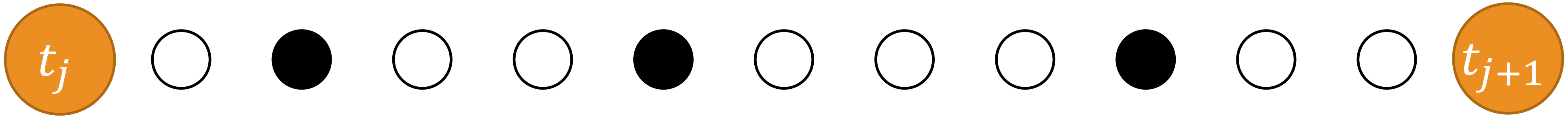
How much can RG lose between t_j and t_{j+1}?

Recall insight 1: if greedy loses at a time step, it removes some actions from best actions.

Round 2 – Randomized Greedy

How much can RG lose between t_j and t_{j+1}?

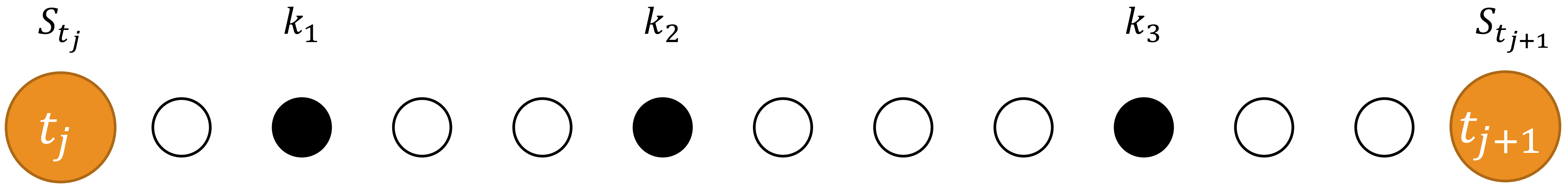
	S_{t_j}	k_1	k_2	k_3	$S_{t_{j+1}}$



# of best actions =	$\left\|S_{t_j}\right\|$	$\left\|S_{t_j}\right\| - k_1$	$\left\|S_{t_j}\right\| - k_1 - k_2$	$\left\|S_{t_j}\right\| - k_1 - k_2 - k_3$	
Loss:		$\dfrac{k_1}{\left\|S_{t_j}\right\|}$	$\dfrac{k_2}{\left\|S_{t_j}\right\| - k_1}$	$\dfrac{k_3}{\left\|S_{t_j}\right\| - k_1 - k_2}$	

Round 2 – Randomized Greedy

How much can RG lose between t_j and t_{j+1}?



Loss: $\dfrac{k_1}{|S_{t_j}|}$ $\quad \dfrac{k_2}{|S_{t_j}| - k_1}$ $\quad \dfrac{k_3}{|S_{t_j}| - k_1 - k_2}$

$$\frac{k_1}{|S_{t_j}|} + \frac{k_2}{|S_{t_j}| - k_1} + \frac{k_3}{|S_{t_j}| - k_1 - k_2} + \cdots + \frac{k_r}{|S_{t_j}| - \sum_{q=1}^{r-1} k_q}$$

Round 2 – Randomized Greedy

How much can RG lose between t_j and t_{j+1}?

$$\underbrace{\frac{k_1}{|S_{t_j}|}}_{\le \frac{1}{|S_{t_j}|}+\frac{1}{|S_{t_j}|-1}+\cdots+\frac{1}{|S_{t_j}|-k_1+1}} + \underbrace{\frac{k_2}{|S_{t_j}|-k_1}}_{\le \frac{1}{|S_{t_j}|-k_1}+\cdots+\frac{1}{|S_{t_j}|-k_1-k_2+1}} + \frac{k_3}{|S_{t_j}|-k_1-k_2} + \cdots + \underbrace{\frac{k_r}{|S_{t_j}|-\sum_{q=1}^{r-1}k_q}}_{\le \frac{1}{|S_{t_j}|-\sum_{q=1}^{\ell-1}k_q}+\cdots+\frac{1}{|S_{t_j}|-\sum_{q=1}^{\ell}k_q+1}}$$

Round 2 – Randomized Greedy

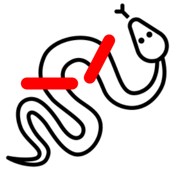
How much can RG lose between t_j and t_{j+1}?

$$\frac{k_1}{|S_{t_j}|} + \frac{k_2}{|S_{t_j}| - k_1} + \frac{k_3}{|S_{t_j}| - k_1 - k_2} + \cdots + \frac{k_r}{|S_{t_j}| - \sum_{q=1}^{r-1} k_q}$$

$$\leq \frac{1}{|S_{t_j}|} + \frac{1}{|S_{t_j}| - 1} + \cdots + \frac{1}{|S_{t_j}| - \sum_{q=1}^{\ell} k_q + 1}$$

$$\leq \frac{1}{n} + \cdots + \frac{1}{|S_{t_j}|} + \frac{1}{|S_{t_j}| - 1} + \cdots + \frac{1}{|S_{t_j}| - \sum_{q=1}^{\ell} k_q + 1} + \cdots + 1$$

$$\leq \log n + 1$$



Round 2 – Randomized Greedy

The RG algorithm can lose at most $\log n + 1$ times (in expectation) for every loss of the best action.

The worst-case “badness” of the RG algorithm is logarithmically bounded by the ”badness” of the best action. Not so bad…

But we can do better!

Round 3 – Multiplicative Weights Updates

"The multiplicative weights algorithm was such a great idea, that it was discovered three times"
– C. Papadimitriou [Seminar Talk]

`Initialize` $w_i^1 \leftarrow 1, p_i^1 \leftarrow \frac{1}{n}$;

At $t = 1, \ldots, \infty$:

For $i = 1, \ldots n$ **do**:

$w_i^t \leftarrow w_i^{t-1} e^{\epsilon v_i^{t-1}}$;

End For

$W_t \leftarrow \sum_i w_i^t$;

For $i = 1, \ldots n$ **do**:

$p_i^t \leftarrow \frac{w_i^t}{W_t}$;

End For

Let w_i^t be the weight of action i at time t, and $W_t = \sum_i w_i^t$.

Let w_b^t be the weight of the best action at time t. Then:

$$W_{t+1} = \sum_i w_i^{t+1} \geq w_b^{t+1} = w_b^t \times e^{\varepsilon v_b^t}$$

$$= w_b^{t-1} \times e^{\varepsilon v_b^{t-1} + \varepsilon v_b^t} = \cdots = 1 \times e^{\varepsilon \sum_{q=1}^t v_b^q} = e^{\varepsilon \times V_b^t}$$

So...

$$W_{t+1} \geq e^{\varepsilon V_b^t}$$

Let’s upper bound W_{t+1} with something that has the MWU gain in it!

Let w_i^t be the weight of action i at time t, and $W_t = \sum_i w_i^t$.

$$W_{t+1} = \sum_i w_i^{t+1} = \sum_i w_i^t \times e^{\varepsilon v_i^t}$$

We assume that $\varepsilon \in [0,1]$ and that $v_i^t \in \{0,1\}$, so $\varepsilon v_i^t \in [0,1]$.

Fact: for any $x \in [-1,1]$, $e^x \leq 1 + x + x^2$. So…

$$\sum_i w_i^t \times e^{\varepsilon v_i^t} \leq \sum_i w_i^t \times \left(1 + \varepsilon v_i^t + \left(\varepsilon v_i^t\right)^2\right) \leq$$

$$\boxed{p_i^t = w_i^t / W_t}$$

$$\leq \sum_i w_i^t \times \left(1 + \varepsilon v_i^t + \varepsilon^2\right) = (1 + \varepsilon^2) \times \sum_i w_i^t + \varepsilon \sum_i w_i^t \times v_i^t$$

$$= (1 + \varepsilon^2) \times W_t + \varepsilon W_t \sum_i p_i^t \times v_i^t = (1 + \varepsilon^2) \times W_t + \varepsilon W_t v_{MWU}^t$$

$$= W_t \times (1 + \varepsilon^2 + \varepsilon v_{MWU}^t) \leq W_t \times e^{\varepsilon^2 + \varepsilon v_{MWU}^t}$$

$$\boxed{1 + x \leq e^x}$$

Let w_i^t be the weight of action i at time t, and $W_t = \sum_i w_i^t$.

$$W_{t+1} \le W_t \times e^{\varepsilon^2 + \varepsilon v_{MWU}^t} \le W_{t-1} \times e^{2\varepsilon^2 + \varepsilon\left(v_{MWU}^t + v_{MWU}^{t-1}\right)} \le \cdots$$
$$\le W_1 \times e^{2\varepsilon^2 + \varepsilon\left(v_{MWU}^t + v_{MWU}^{t-1} + \cdots + v_{MWU}^1\right)} = W_1 \times e^{t\varepsilon^2 + \varepsilon V_{MWU}^t}$$
$$= n \times e^{t\varepsilon^2 + \varepsilon V_{MWU}^t}$$

To conclude:

$$e^{\varepsilon V_b^t} \le W_{t+1} \le n \times e^{t\varepsilon^2 + \varepsilon V_{MWU}^t}$$

Taking logs of both sides we get:

$$\varepsilon V_b^t \le \log n + t\varepsilon^2 + \varepsilon V_{MWU}^t \Rightarrow V_b^t \le \frac{\log n}{\varepsilon} + t\varepsilon + V_{MWU}^t$$

Dividing both sides by t:

$$\frac{V_b^t}{t} \le \frac{\log n}{\varepsilon t} + \varepsilon + \frac{V_{MWU}^t}{t} \Leftrightarrow$$

Randomized Greedy:

$$V_b^t \le \frac{\log n + 1}{t} + (\log n + 1)\frac{V_{MWU}^t}{t}$$

Discussion Points

- This entire section can easily be translated to negative rewards
- Rewards don't have to be in $\{0,1\}$; can be any real numbers in a range $[a, b]$: but if $b - a$ is very large, affects learning rate.
- A lot of the things here generalize to multi-armed bandits (need to devote more time to exploration – affects learning rate), but the same idea of the MWU algorithm holds:
  - Place exponential weights on actions
  - Perhaps change ε as a function of time